

Advanced Optimization

Chapter 2 — Fundamentals of Unconstrained Optimization

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 2

The unconstrained problem

$$\min_{x \in \mathbb{R}^n} f(x), \quad f : \mathbb{R}^n \rightarrow \mathbb{R} \text{ smooth.} \quad (2.1)$$

What we know about f is *local*: values, and perhaps derivatives, at the points $x_0, x_1, x_2, \dots$ that our algorithm chooses to visit. There is no global picture.

Two consequences that shape everything:

- ▶ The algorithm decides where to look. Good algorithms look in places that identify a solution reliably without wasting evaluations.
- ▶ Information about f is often expensive — so we prefer methods that ask for no more of it than necessary (values only? gradients? Hessians?).

A motivating example: nonlinear least squares

Fit measurements y_j at times t_j with a model $\phi(t; x) = x_1 + x_2 e^{-(x_3 - t)^2 / x_4} + x_5 \cos(x_6 t)$. Define residuals $r_j(x) = y_j - \phi(t_j; x)$ and solve

$$\min_{x \in \mathbb{R}^6} f(x) = r_1^2(x) + \cdots + r_m^2(x). \quad (2.3)$$

- ▶ Only $n = 6$ unknowns, but if $m \sim 10^5$ each evaluation of f is expensive. **Dimension and cost are different axes.**
- ▶ Suppose a solver returns x^* with $f(x^*) = 0.34 \neq 0$. The model does not reproduce the data exactly. **How do we know x^* is a minimizer at all?**

That question — what is a solution, and how do we recognize one — is the content of §2.1. The least-squares structure itself returns in Chapter 10.

Four kinds of minimizer

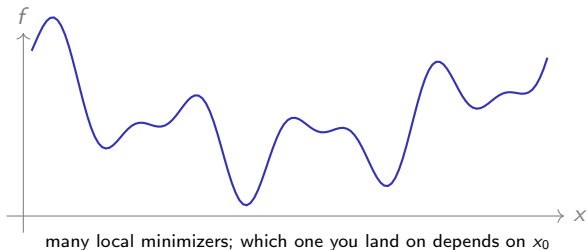
- ▶ **Global minimizer:** $f(x^*) \leq f(x)$ for all $x \in \mathbb{R}^n$.
- ▶ **Local (weak) minimizer:** there is a neighborhood $\mathcal{N}$ of x^* with $f(x^*) \leq f(x)$ for all $x \in \mathcal{N}$.
- ▶ **Strict (strong) local minimizer:** $f(x^*) < f(x)$ for all $x \in \mathcal{N}$, $x \neq x^*$.
- ▶ **Isolated local minimizer:** x^* is the *only* local minimizer in some $\mathcal{N}$.

Examples

- ▶ $f(x) = 2$: every point is a weak local minimizer, none is strict.
- ▶ $f(x) = (x - 2)^4$: strict local minimizer at $x = 2$.
- ▶ $f(x) = x^4 \cos(1/x) + 2x^4$, $f(0) = 0$: strict local minimizer at 0, but **not isolated** — there are strict local minimizers $x_j \rightarrow 0$.

Isolated $\Rightarrow$ strict. The converse fails, as the last example shows.

Why the global minimizer is usually out of reach



- ▶ Our knowledge of f is local; we cannot rule out a sharp dip in an unsampled region.
- ▶ Algorithms tend to be *trapped* at a local minimizer. This is not pathological: molecular-conformation potentials have millions of local minima.
- ▶ The one important exception: for **convex** f , every local minimizer is global.

Recognizing a local minimum

From the definitions alone, checking that x^* is a local minimizer seems to require examining every point in a neighborhood.

For smooth f there is a far better way: look only at $\nabla f(x^*)$ and $\nabla^2 f(x^*)$.

The tool that makes this possible — and that underlies essentially every argument in this book — is **Taylor's theorem**. We state it once, carefully, and then lean on it for the rest of the semester.

Taylor's theorem (Theorem 2.1)

For f continuously differentiable and $p \in \mathbb{R}^n$:

$$f(x + p) = f(x) + \nabla f(x + tp)^T p, \quad \text{some } t \in (0, 1). \quad (2.4)$$

If f is twice continuously differentiable:

$$\nabla f(x + p) = \nabla f(x) + \int_0^1 \nabla^2 f(x + tp) p \, dt, \quad (2.5)$$

$$f(x + p) = f(x) + \nabla f(x)^T p + \frac{1}{2} p^T \nabla^2 f(x + tp) p, \quad \text{some } t \in (0, 1). \quad (2.6)$$

These are equalities, not approximations

This is the mean-value form, not the truncated series you may remember. The price is an unknown $t \in (0, 1)$; the reward is that *bounding the term containing t* turns an equality into a rigorous inequality. Every convergence proof in this book is built that way.

Homework: derive all three from the one-variable function $g(t) = f(x + tp)$.

First-order necessary condition (Theorem 2.2)

If x^* is a local minimizer and f is continuously differentiable near x^* , then $\nabla f(x^*) = 0$.

Why (the shape of the argument, not the full proof): suppose $\nabla f(x^*) \neq 0$ and set $p = -\nabla f(x^*)$. Then $p^T \nabla f(x^*) < 0$, and by continuity $p^T \nabla f(x^* + tp) < 0$ for small t . By (2.4), $f(x^* + \bar{t}p) = f(x^*) + \bar{t} p^T \nabla f(x^* + \bar{t}p) < f(x^*)$ — we have found a direction of decrease, so x^* was not a local minimizer.

Any point with $\nabla f(x^*) = 0$ is called a **stationary point**. Every local minimizer is stationary; **the converse is false**, as we will see.

Second-order conditions (Theorems 2.3, 2.4)

Second-order necessary

If x^* is a local minimizer and $\nabla^2 f$ is continuous near x^* , then $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succeq 0$.

Why: if $p^T \nabla^2 f(x^*) p < 0$ for some p , then (2.6) gives

$f(x^* + \bar{t}p) = f(x^*) + \frac{1}{2} \bar{t}^2 p^T \nabla^2 f(x^* + tp) p < f(x^*)$ — a direction of *negative curvature* along which f decreases.

Second-order sufficient

If $\nabla f(x^*) = 0$ and $\nabla^2 f(x^*) \succ 0$, then x^* is a *strict* local minimizer.

Why: positive definiteness persists in a ball around x^* ; then (2.6) gives

$f(x^* + p) = f(x^*) + \frac{1}{2} p^T \nabla^2 f(z) p > f(x^*)$ for every small $p \neq 0$.

Note the sufficient condition delivers *more* (strictness) than the necessary one demands.

The gap between necessary and sufficient

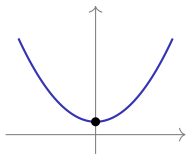
	Necessary	Sufficient
Gradient	$\nabla f(x^*) = 0$	$\nabla f(x^*) = 0$
Hessian	$\nabla^2 f(x^*) \succeq 0$	$\nabla^2 f(x^*) \succ 0$

They do not meet. $f(x) = x^4$: $x = 0$ is a strict local minimizer, yet $f''(0) = 0$ — the sufficient condition fails. Sufficiency is not necessary.

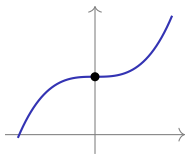
Why this gap matters for algorithms

An algorithm can verify $\nabla f \approx 0$ and check the sign of $\nabla^2 f$. When the Hessian is singular or indefinite, *the conditions cannot tell it what it has found*. Indefinite Hessians are the central practical difficulty of nonconvex optimization; trust regions (Ch. 4), Hessian modification (Ch. 3), and inertia correction in interior-point methods (Ch. 19) all exist because of this gap.

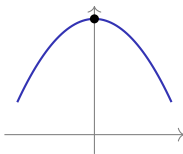
What a stationary point can be



$\nabla^2 f \succ 0$
local minimum



$\nabla^2 f$ indefinite / singular
saddle point



$\nabla^2 f \prec 0$
local maximum

All three have $\nabla f = 0$. Every algorithm in this course drives $\|\nabla f\| \rightarrow 0$; **none can promise which of the three you land on**, and none can promise a global minimizer. In $\mathbb{R}^n$ with n large, saddle points vastly outnumber minima.

The convex case (Theorem 2.5) — Boyd's world in one theorem

If f is convex, any local minimizer is a global minimizer. If moreover f is differentiable, any stationary point is a global minimizer.

Why: if z does better than x^* , then by convexity every point on the segment from x^* to z does better too — and that segment enters every neighborhood of x^* . For the second part, the directional derivative toward z is $\leq f(z) - f(x^*) < 0$, so $\nabla f(x^*) \neq 0$.

This is the theorem that made the prerequisite course work: stationarity was the whole story.

Without convexity, stationarity is where the story starts.

One line summarizes §2.1: all algorithms, in one way or another, seek a point where ∇f vanishes. What they then do about second-order information is what separates them.

Nonsmooth problems — out of scope, with one remark

- ▶ This book assumes $f \in C^2$: second derivatives exist and are continuous.
- ▶ Discontinuous f : no general theory; if f is piecewise smooth, minimize each piece.
- ▶ Continuous but nondifferentiable (a kink at the minimum): the subgradient replaces the gradient. *Difficult*, because information at one point no longer predicts behavior at neighbors.

The remark that will matter later

Certain nonsmooth objectives, notably

$$f(x) = \|r(x)\|_1 \quad \text{and} \quad f(x) = \|r(x)\|_\infty,$$

can be reformulated as *smooth constrained* problems (Ch. 12 exercises, Ch. 17). They are useful in data fitting, and the ℓ_1 form reappears as a *merit function* in SQP and interior-point methods.

Iterates and monotonicity

All algorithms here: start from x_0 (user-supplied if possible), generate $\{x_k\}_{k \geq 0}$, terminate when progress stops or accuracy is sufficient.

To choose x_{k+1} they use information about f at x_k , and possibly at $x_0, \dots, x_{k-1}$, to find a point with *lower* function value.

Monotone vs. nonmonotone

Most methods insist on $f(x_{k+1}) < f(x_k)$ at every step. *Nonmonotone* methods allow occasional increases but still require $f(x_k) < f(x_{k-m})$ after some fixed number m of steps. (This flexibility returns in Ch. 15 and in the filter methods of Ch. 18–19.)

There are exactly **two fundamental strategies** for the move $x_k \rightarrow x_{k+1}$. Nearly every algorithm in this book follows one of them.

Strategy 1: line search

Choose a **direction** p_k first, then a **step length** α by approximately solving

$$\min_{\alpha > 0} f(x_k + \alpha p_k). \quad (2.10)$$

- ▶ Solving (2.10) exactly extracts the most from p_k — but is expensive and **usually unnecessary**.
- ▶ Instead: generate a few trial step lengths and accept one that *loosely* approximates the minimizer. What “loosely” must mean is the subject of Ch. 3.
- ▶ At the new point, compute a new direction and repeat.

Direction first, distance second.

Strategy 2: trust region

Build a **model** m_k that behaves like f near x_k ; since it may be poor far away, restrict the search to a region around x_k :

$$\min_p m_k(x_k + p) \quad \text{s.t. } x_k + p \text{ inside the trust region.} \quad (2.11)$$

Usually the region is a ball $\|p\|_2 \leq \Delta_k$; Δ_k is the **trust-region radius**.

The model is almost always quadratic,

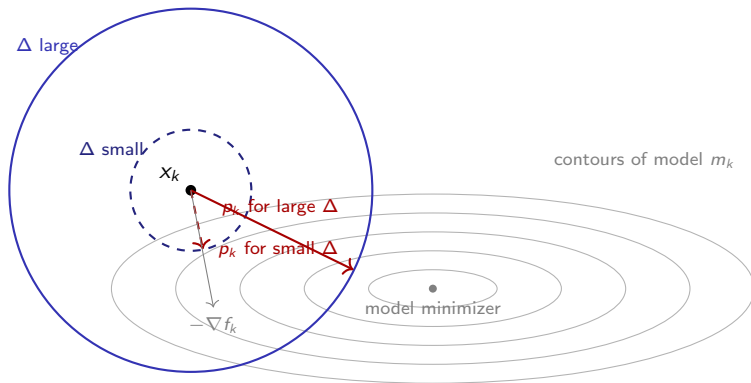
$$m_k(x_k + p) = f_k + p^T \nabla f_k + \frac{1}{2} p^T B_k p, \quad (2.12)$$

agreeing with f to first order at x_k ; B_k is $\nabla^2 f_k$ or an approximation.

If the candidate step does not produce sufficient decrease in f : the region was too large — **shrink Δ_k and re-solve**.

Distance first, direction second.

Shrinking the region changes the direction (Figure 2.4)



After a rejected step the next candidate is **shorter and points in a different direction**. A line search would stay on the same ray. This is why trust-region methods cope naturally with indefinite B_k , and line-search methods must repair it first.

The two strategies, side by side

	Line search	Trust region
Fix first	direction p_k	distance Δ_k
Then choose	step length α_k	direction and length within Δ_k
On failure	shorten α , same ray	shrink Δ_k , re-solve; direction changes
Model	used to get p_k	minimized directly, subject to $\ p\ \leq \Delta_k$
Indefinite B_k	must be modified	handled by the constraint
Detailed in	Chapter 3	Chapter 4

Two issues to preview now, because they are closely related: the choice of **search direction** p_k in line-search methods, and the choice of B_k in trust-region methods.

Steepest descent

Which unit direction decreases f fastest at x_k ? By (2.6),

$$f(x_k + \alpha p) = f(x_k) + \alpha p^T \nabla f_k + \frac{1}{2} \alpha^2 p^T \nabla^2 f(x_k + tp) p,$$

so the rate of change along p is $p^T \nabla f_k$. Minimize it:

$$\min_p p^T \nabla f_k \quad \text{s.t. } \|p\| = 1. \quad (2.13)$$

Since $p^T \nabla f_k = \|\nabla f_k\| \cos \theta$, the minimum is at $\cos \theta = -1$:

$$p = -\nabla f_k / \|\nabla f_k\|.$$

- ▶ Orthogonal to the contours of f . Needs only the gradient.
- ▶ Can be excruciatingly slow on badly conditioned problems — see the scaling slide.

More generally, any **descent direction** — angle strictly less than $\pi/2$ with $-\nabla f_k$, i.e. $p^T \nabla f_k < 0$ — decreases f for small enough steps.

Newton direction

Minimize the second-order model instead:

$$f(x_k + p) \approx f_k + p^T \nabla f_k + \frac{1}{2} p^T \nabla^2 f_k p =: m_k(p). \quad (2.14)$$

Assuming $\nabla^2 f_k \succ 0$, setting $\nabla m_k(p) = 0$ gives

$$p_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k. \quad (2.15)$$

- ▶ Compare (2.14) with (2.6): the only difference is $\nabla^2 f(x_k + tp)$ replaced by $\nabla^2 f_k$ — an $O(\|p\|^3)$ perturbation, so the model is accurate for small p .
- ▶ Natural step length $\alpha = 1$. Use it when it works; adjust only when it does not.
- ▶ Descent direction when $\nabla^2 f_k \succ 0$: $\nabla f_k^T p_k^N = -(p_k^N)^T \nabla^2 f_k p_k^N < 0$.
- ▶ Local convergence typically **quadratic**: a few iterations to high accuracy once in the neighborhood.

Two costs

- (i) When $\nabla^2 f_k \not\succ 0$, p_k^N may be undefined or fail to be a descent direction — must be modified (Ch. 3). (ii) The Hessian itself may be cumbersome, error-prone, and expensive (Ch. 8).

Quasi-Newton: the secant equation

Idea: avoid the Hessian but keep superlinear convergence, by *learning* curvature from gradient changes. From (2.5), adding and subtracting $\nabla^2 f(x)p$,

$$\nabla f(x+p) = \nabla f(x) + \nabla^2 f(x)p + \int_0^1 [\nabla^2 f(x+tp) - \nabla^2 f(x)]p \, dt,$$

and the integral is $o(\|p\|)$ by continuity. With $x = x_k$, $p = x_{k+1} - x_k$, near a solution the linear term dominates:

$$\nabla^2 f_k(x_{k+1} - x_k) \approx \nabla f_{k+1} - \nabla f_k. \quad (2.16)$$

Require the approximation B_{k+1} to mimic this exactly — the **secant equation**:

$$B_{k+1}s_k = y_k, \quad s_k = x_{k+1} - x_k, \quad y_k = \nabla f_{k+1} - \nabla f_k. \quad (2.17)$$

One equation, n^2 unknowns. Add **symmetry** and a **low-rank change** from B_k to B_{k+1} to pin it down. Two answers dominate the field.

Quasi-Newton: SR1 and BFGS

Symmetric rank-one (SR1):

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k)(y_k - B_k s_k)^T}{(y_k - B_k s_k)^T s_k}. \quad (2.18)$$

BFGS (Broyden–Fletcher–Goldfarb–Shanno), rank two:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}. \quad (2.19)$$

Both satisfy (2.17) and preserve symmetry. BFGS stays positive definite whenever $B_0 \succ 0$ and $s_k^T y_k > 0$ — a condition the line search will have to guarantee (Ch. 3, Ch. 6).

Search direction: $p_k = -B_k^{-1} \nabla f_k$ (2.20). In practice update the inverse $H_k = B_k^{-1}$ directly,

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = 1/(y_k^T s_k), \quad (2.21)$$

so $p_k = -H_k \nabla f_k$ is a matrix–vector product, no factorization. Large-scale variants (limited memory, partial separability): Ch. 7.

Nonlinear conjugate gradient

$$p_k = -\nabla f(x_k) + \beta_k p_{k-1},$$

with the scalar β_k chosen so that p_k and p_{k-1} are *conjugate* — a notion from quadratic minimization, defined properly in Ch. 5.

- ▶ Origin: solving $Ax = b$ with A symmetric positive definite, equivalently minimizing $\phi(x) = \frac{1}{2}x^T Ax - b^T x$. Extension to general f was natural.
- ▶ Much more effective than steepest descent, almost as cheap.
- ▶ Does not reach Newton or quasi-Newton rates — but **stores no matrix at all**. The default at very large scale, and the engine inside Newton-CG (Ch. 7) and the trust-region subproblem solver (Ch. 4).

All four directions work inside a line search. All but CG have a trust-region analogue.

The rates just mentioned, defined (Appendix A.2)

Let $\{x_k\} \rightarrow x^*$. Convergence is

- ▶ **Q-linear** if $\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \leq r$ for some $r \in (0, 1)$ and all large k ;
- ▶ **Q-superlinear** if $\lim_{k \rightarrow \infty} \frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} = 0$;
- ▶ **Q-quadratic** if $\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} \leq M$ for some $M > 0$ and all large k .

Q = quotient. The weaker **R-rates** (root) only require the error to be *dominated* by a sequence with the corresponding Q-rate; an R-linear sequence may increase its error on some iterations. The book drops the “Q” and says linear, superlinear, quadratic.

Steepest descent	linear	$r \rightarrow 1$ when ill-conditioned
Quasi-Newton	superlinear	
Newton	quadratic	under appropriate assumptions

These are *local* rates: they describe the endgame, not the journey.

Models for trust-region methods

Same quadratic (2.12); the choice of B_k decides the method.

- ▶ $B_k = 0$, Euclidean ball: the subproblem is $\min_p f_k + p^T \nabla f_k$ s.t. $\|p\|_2 \leq \Delta_k$, solved in closed form by

$$p_k = -\Delta_k \nabla f_k / \|\nabla f_k\|.$$

Steepest descent with the step set by the radius — line search and trust region coincide.

- ▶ $B_k = \nabla^2 f_k$: **trust-region Newton**. Because of $\|p\|_2 \leq \Delta_k$, **the subproblem has a solution even when $\nabla^2 f_k$ is indefinite** (Figure 2.4). Highly effective in practice; Ch. 7.
- ▶ B_k a quasi-Newton approximation: **trust-region quasi-Newton**.

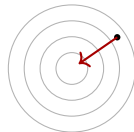
This is the structural reason trust regions were introduced: they turn the “indefinite Hessian” problem from an obstacle into a constrained subproblem.

Scaling

A problem is **poorly scaled** if changes to x in one direction produce far larger changes in f than in another. Example: $f(x) = 10^9 x_1^2 + x_2^2$.



poorly scaled: $-\nabla f_k$ nearly misses



well scaled: $-\nabla f_k$ points at the minimizer

- ▶ Arises naturally when variables have wildly different magnitudes (e.g. rate constants 10^{-10} and 10^5 in the same model), or when units change.
- ▶ Remedy: **diagonal scaling**, $x = Dz$ with D chosen so that z is $O(1)$.
- ▶ **Steepest descent is sensitive to scaling; Newton's method is not** — the quadratic model already captures the geometry. Prefer algorithms that are scale-invariant.

Chapter 2 in one table

Direction / model	Uses	Local rate	Chapter
Steepest descent	∇f	linear	3
Newton	$\nabla f, \nabla^2 f$	quadratic	3, 7
Quasi-Newton	∇f , secant updates	superlinear	6, 7
Conjugate gradient	∇f , previous direction	linear ⁺	5
Line search	direction $\rightarrow$ step length	—	3
Trust region	radius $\rightarrow$ step	—	4

What to carry forward

- ▶ Taylor's theorem in mean-value form, and the habit of bounding the unknown- t term.
- ▶ The gap between necessary and sufficient conditions — the home of indefinite Hessians.
- ▶ Two strategies, four directions, one quadratic model.
- ▶ The secant equation: curvature can be learned from gradients.

Homework (derivations)

- 1 Derive the three forms of Taylor's theorem (2.4)–(2.6) from the scalar function $g(t) = f(x + tp)$, $t \in [0, 1]$, using the mean value theorem and the fundamental theorem of calculus.
- 2 Show that the solution of (2.13) is $p = -\nabla f_k / \|\nabla f_k\|$.
- 3 Show that the Newton direction (2.15) is a descent direction whenever $\nabla^2 f_k \succ 0$, and give a 2×2 example where $\nabla^2 f_k$ is indefinite and p_k^N is *not* a descent direction.
- 4 Verify that both SR1 (2.18) and BFGS (2.19) satisfy the secant equation (2.17).
- 5 Prove that $1 + k^{-k}$ converges superlinearly to 1, and that $1 + (0.5)^{2^k}$ converges quadratically.

Next: N&W §3.1–3.3 — *how far along p_k ?*